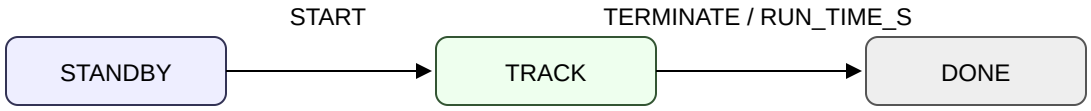
propPitch: how it works

Holds a propeller plane's collective blade pitch at the angle that gives the most forward thrust, computed every tick from the game's own blade aerodynamics, and runs the rotors at the RPM limit the throttle sets. A throttle cut stops and feathers the props for the glide. With Brakes on, the props brake the plane: on the wheels held to the thrust that keeps every wheel down, and stopped once the plane is still. The props' reaction torque on the airframe is cancelled with the trims. It flies only the propellers; the pilot flies the plane. START works the same on the runway or in flight.

Phases

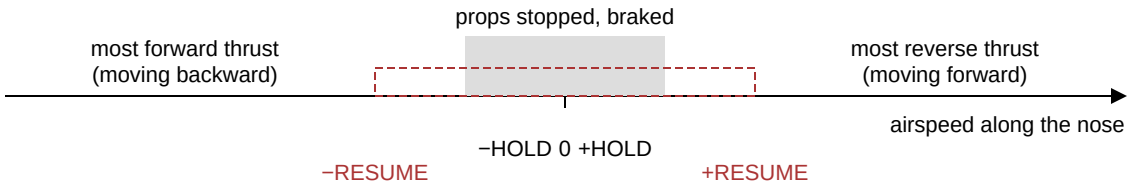


START finds every robotic rotor and the control-surface blades below it (any part with a `Deploy Angle` field), disables the craft's KAL controllers (`DISABLE_KAL`), runs every rotor at the throttle's RPM limit and `ROTOR_TORQUE_PCT` with its motor engaged and unlocked, releases each rotor's brake, and reads once, in the vessel frame: each blade's position and axes (relative to its rotor), each rotor's hub, and each wheel's bounding box. It records the pilot's pitch/roll/yaw trim. Blade part names are looked up in `bladeTable.py`; an unknown blade stops START with its name. TRACK reads everything in one batch, sets the rotors' RPM limit and commands the blades every tick; every `REASSERT_S` it re-asserts the KALs' and rotors' settings and the brakes. TERMINATE re-enables the KALs, restores each rotor's RPM and torque limits and motor state, and puts the pilot's own trim back.

Throttle sets the RPM limit

- With `THROTTLE_RPM` on, RPM limit = max(`THROTTLE_RPM_MIN`, throttle × `ROTOR_RPM`) (460, 0). The throttle is `vessel.control.throttle`, whatever drives it.
- The rotors' `target_rpm` is written only when that moved by `RPM_WRITE_STEP` (2 rpm) or reached an end of the range; the re-assert holds the rotors to the last value written.
- The pitch search scores every angle at that RPM limit.
- **Feather:** a throttle at or below `THROTTLE_FEATHER` (0.02) is a cut: the limit is 0 and the rotors are held stopped. At 0 rpm the most thrust is the least drag, so the search, over the blades' whole Deploy Angle range, puts every blade edge-on to the flow (Deploy 0 in axial flow). The re-assert's brake release is skipped while the limit is at or below `BRAKE_RPM`.
- `THROTTLE_RPM=False` runs every rotor at `ROTOR_RPM` whatever the throttle.

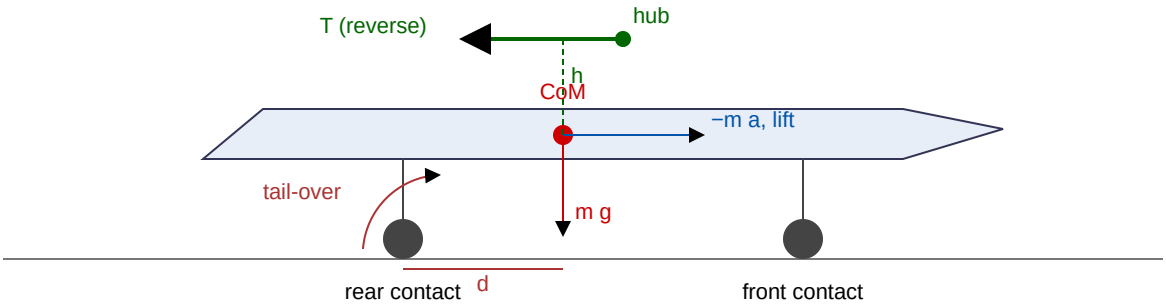
Prop brake (the Brakes group)



What the props do while `vessel.control.brakes` is on (`PROP_BRAKE`). Grey: entered inside ±`BRAKE_HOLD_SPEED` (0.5 m/s); left only past ±`BRAKE_RESUME_SPEED` (2 m/s, dashed).

- **Moving:** every rotor at `ROTOR_RPM` whatever the throttle, its own brake latch (which the craft's Brakes group sets) released on the press and at every re-assert. The sign of the air velocity's nose component (vessel y) picks the objective: forward → the angle with the most thrust against the nose, backward → the most along it. The search runs over the blades' whole Deploy Angle range, ±1.5 × ctrlSurfaceRange of the tightest blade (±90° for fan blades and propellers, ±22.5° for heli blades; `bladeTable.py`'s `range`); negative angles give reverse thrust. On the wheels the thrust is bounded (next section).
- **Still:** once |v| < `BRAKE_HOLD_SPEED`, every rotor's limit goes to 0 rpm and its brake latch is set (the rotor's `BrakeAction`, activated); the blades are left where they are. It stays stopped until |v| > `BRAKE_RESUME_SPEED`, which releases the latches and spins the props up again. The log says `prop brake: still at … props stopped` / `moving at … props spun up`; `mode=brake-stop`.
- Brakes off returns to the throttle's RPM and the forward optimum on the next tick (the group's own action releases the latches, and the re-assert releases any rotor left stopped).

The tip limit (on the wheels)



Reverse thrust from props above the CoM lifts the front wheels about the rear contact; gravity's moment m g d holds them down. Props below the CoM tip it the other way, about the front contact (a nose-over).

While the props brake and a wheel touches (or has touched during this brake), the braking optimum is held to a bound from `tipping.thrust_bounds` (`BRAKE_TIP_LIMIT`):

- **Contacts:** for every wheel grounded since Brakes went on (`Wheel.grounded`), the bottom of its bounding box along the local up (`transform_direction` of the surface frame's up into the vessel frame). A wheel lifting is the tip starting, not new geometry, so it stays a contact. A bounce with every wheel up keeps the limit on.
- **Forces** (vessel frame, about the CoM): gravity −m g up; the measured aero force (`Flight.aerodynamic_force`) less the props' thrust, at the CoM (its vertical part is lift); the

deceleration that aero force alone gives along the ground, as an inertial force −m a at the CoM; the props' thrust T along the nose at the thrust-weighted mean hub. The ground's forces act on the contact line and have no moment about it.

- **Bound:** about the rearmost and the foremost contact, the moment that lifts the far wheels is linear in T. Moving T moves the deceleration with it, so together they move the moment by T·(hub × nose): the thrust line's height h over the CoM. The pilot solves for the T that leaves `BRAKE_TIP_MARGIN` (0.5) of gravity's restoring moment m g d. Rear pivot → a floor on reverse thrust, front pivot → a ceiling on forward thrust. Past the balance point (the CoM beyond the pivot) the bound is the balance itself. A floor is never above 0, a ceiling never below.
- The props' own thrust cancels out of the bound, so it does not depend on the model's thrust or on a derivative. The wheel brakes' moment is not counted: with props above the CoM it works against a tail-over; with props below it, it adds to a nose-over.
- **Hill climb (backstop):** the bound is multiplied by a fraction that halves (at most once per `BRAKE_TIP_BACKOFF_S`, 0.3 s) when a contact wheel has been off the ground `BRAKE_TIP_LIFT_S` (0.25 s) or the pitch moves faster than `BRAKE_TIP_RATE_DEG_S` (10°/s), and rises by `BRAKE_TIP_RISE_PER_S` (0.25/s) while quiet. A tick that cannot compute keeps the last bound. Where no bound was ever computed (the contacts do not span 0.3 m along the ground: one wheel, or one row across), the climb is the whole limit: from no thrust, up by `BRAKE_TIP_CLIMB_G_PER_S` (0.1 g/s) of m g, halved the same way.
- **Search:** if the free optimum passes the bound, the search runs again with the score folded at the bound (past it the score falls), so it returns the most thrust that does not pass it. Airborne the brake is unlimited.

Reaction-torque counter

The motor turns its blades against their aero load q; the airframe feels −q about the rotor's real spin direction. Rotors that pair off cancel; whatever is left is cancelled with the trims (`TORQUE_COUNTER`):

- **Spin sense:** kRPC does not say which way a rotor turns, and the model's thrust depends only on the product of spin and blade deflection sense. Once a rotor turns faster than `SPIN_PROBE_RPM` (30), each tick reads one of its blades' positions. The blade's angle about the hub since the previous tick is compared with what +rpm·dt and −rpm·dt would have turned it by (modulo a turn). A tick votes only when the two predictions differ by 40° or more; the sense is decided when one side leads by `SPIN_VOTES` (5). Until every rotor is known there is no counter. Event `rotor spin senses (x model)`.
- **Torque:** $\tau = \Sigma\ -q_i \cdot s_i \cdot \text{axis}_i$, with q from the model at the angle flown (where each rotor settles) and s_i the measured sense. Zero while the props are stopped.
- **Input:** KSP applies an input u as ReferenceTransform·(−u × torque) per axis (`ModuleReactionWheel.ActiveUpdate`; control surfaces answer the same way), so the input that cancels τ is τ divided by kRPC's `available_torque` on that axis, in the direction needed (pitch = x, roll = y, yaw = z), clamped to ±1.
- **Trim:** it is written as pitch/roll/yaw *trim* (persistent, added to the pilot's stick) on top of the pilot's own trim, when it moves by `TRIM_WRITE_STEP` (0.01). A trim read back that matches neither of the last two writes was moved by the pilot, and becomes their trim. TERMINATE restores it. Log `trim=` (the counter's share).

The blade law (KSP's, evaluated off-line)

KSP treats each blade as one lifting surface evaluated at one point: the part's `velocityOffset` along its span (−5.8 m on the R-25, about 6.0 m from the hub). With u the direction of that point's velocity through the air, n the blade's normal turned by s·Deploy Angle about its span, c = u·n, q = ½ρW² and M = W/a at that point:

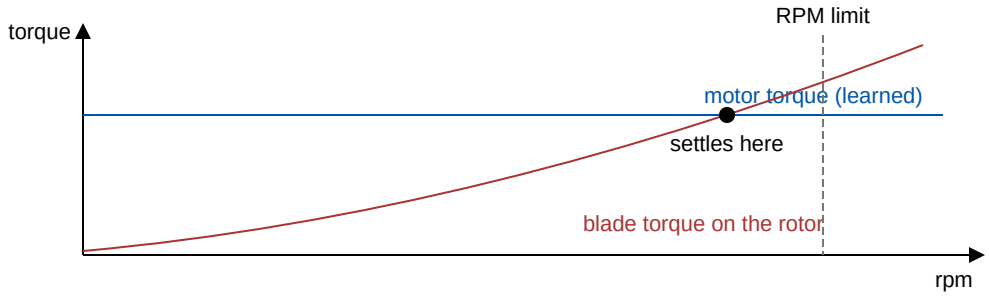
```
lift = −n·sign(c)·Cl(|c|)·Ml(M)·k·q·0.036   (at the part's CoL offset)
drag = −u·Cd(|c|)·Md(M)·k·q·0.015           (at the part's origin)
```

k = deflectionLiftCoeff × ctrlSurfaceArea; the curves are the part cfg's (cubic Hermite FloatCurves), 0.036 and 0.015 Physics.cfg's multipliers. Lift acts along the normal. The point's velocity is the vessel's air velocity (vessel axes, through a hybrid frame) plus ω×r. Each rotor's evenly spaced blades at any one moment stand for the average around the disc. **Deflection sense s:** at START each blade gets the sign that gives forward thrust at rest (60/75/85°) at `ROTOR_RPM`.

The search

Every tick the pilot totals the thrust along the nose at Deploy Angles `PITCH_MIN_DEG`… `PITCH_MAX_DEG` (0…90; braking and feathered, the blades' whole range) on a 1° grid (`SEARCH_STEP_DEG`), then runs a golden-section search around the best cell to `SEARCH_TOL_DEG`. The angle is commanded on every blade when it moved by 0.01°; KSP applies a Deploy Angle without actuator delay.

Load and rpm



Each angle is scored at the rpm its rotor would settle at: the RPM limit if the motor holds it, else where the torques balance.

KSP's rotor is a velocity drive (damper 1000) whose torque is capped at maxTorque × torque limit (70 kN·m on the EM-32S), zero without electric charge or with the brake on; its charge draw, 0.06 × 70 × limit × rpm/460 / 0.75 EC/s, does not depend on load. If the blades' torque at the limit exceeds the motor's, the model bisects for the rpm where they balance; if the air drives the rotor harder than the motor can hold it back, it bisects above the limit (overspeed). The motor torque starts at the cap and is learned (`Rotor.observe`): more than 2% below the limit with a readout flat for 0.6 s, the model's blade torque at that rpm becomes the motor torque (smoothed over 0.5 s); holding the limit, it stays at least the current load and recovers toward the cap over 5 s.

Rotor brakes

A rotor's brake is a latched mode in the game's rotor module: the Brakes action group sets and clears it, and the rotor also latches it when it runs out of charge. A braked rotor holds 0 rpm whatever its motor is told; only the rotor's own `BrakeAction` reads through: Deactivate releases it (the vessel's wheel brakes are left alone), Activate sets it. At START every rotor's brake is released. At each re-assert, a rotor below `BRAKE_RPM` while the Brakes group is off and the limit is above `BRAKE_RPM` is released (a power-loss latch). While the prop brake runs, the re-assert releases every rotor unless the props are stopped.

The log

`pitch= / opt=`	the angle flown / the model's optimum (within the tip bound)
`J=`	measured forward aero force per unit mass (thrust less airframe drag), m/s²
`model=`	the model's blade thrust per unit mass at the angle flown
`rpm=a/b`	measured mean rpm / the rpm the model expects the slowest rotor to settle at
`torque= / motor=`	the worst rotor's blade torque over the cap / learned motor torque per rotor, kN·m
`va=`	velocity through the air in the vessel's axes (x right, y nose, z down)
`thr=`	the throttle read this tick
`mode=`	thrust, feather (throttle cut), brake-rev / brake-fwd / brake-stop (Brakes on)
`tip=`	the tip bound in force (kN) and the hill climb's fraction, or "(climb n g)" where no bound was computed; "-" off the ground or not braking
`trim=`	the counter's share of the pitch/roll/yaw trim
`events`	`wheels n` (at START), `rotor spin senses`, `prop brake on/off`, `still / moving`, `tip: …, braking ×k`, `re-set: …`

`tools/propfly.py`: `--throttle T` holds the throttle; `--brake-at V` presses Brakes past V m/s; `--cut-at V` cuts the throttle past V; `--wheel-brakes P` sets every wheel's brake force as Brakes goes on. `tools/brakesum.py` reads a brake or cut flight: the speed, time to stand still, worst sideslip and nose-up, height gained, tightest bound, back-offs. `tools/bladeprobe.py` / `probesum.py` sweep the angle and compare J with the model.